Seminar 1 - Fundamente JavaScript (ES6+)

Cuprins

1. Tipuri de date, declaratii si Scope
2. Operatori si structuri de control
3. Functii si Arrow Functions
4. Obiecte si Arrays
5. Destructuring si Spread/Rest Operator
6. Metode pentru Arrays
7. Optional Chaining si Nullish Coalescing
8. JSON si date de la API
9. Asincronism si Promises
10. `async` / `await` si tratarea erorilor
11. Module ES6

Exemplele din seminar folosesc un context de **magazin online**.

1. Tipuri de date, declaratii si Scope

Variabile

In JavaScript modern se folosesc in principal `const` si `let`.

```
const storeName = "TechMarket";
let isOnline = true;
let totalProducts = 120;
```

`const` nu permite reasignarea variabilei:

```
const name = "Laptop";

// name = "Phone"; // Error
```

`let` permite modificarea valorii:

```
let quantity = 10;

quantity = 15;
```

`var` este sintaxa mai veche si, in codul modern, este in general inlocuita de `let` si `const`.

Tipuri de date

```
const name = "Laptop";        // string
const price = 3999.99;        // number
const available = true;       // boolean
const value = null;           // null
let discount;                 // undefined
```

Tipuri compuse:

```
const product = {
    name: "Laptop",
    price: 3999
};

const products = [
    "Laptop",
    "Phone",
    "Tablet"
];
```

Scope

Scope-ul determina unde poate fi accesata o variabila.

```
let storeName = "TechMarket";

if (true) {
    let category = "Electronics";

    console.log(category);
}

console.log(storeName);
```

`category` exista doar in blocul `if`.

`let` si `const` au **block scope**.

2. Operatori si structuri de control

Aceasta sectiune este o recapitulare rapida a sintaxei necesare pentru restul seminarului.

Operatori

```
const total = price * quantity;

const finalPrice = price - discount;
```

Comparatii:

```
price > 1000
price >= 1000
price < 5000
price <= 5000

price === 1000
price !== 1000
```

Este recomandat sa folosim `===` in loc de `==`:

```
5 == "5"      // true
5 === "5"     // false
```

Operatori logici:

```
price > 1000 && available

category === "Laptop" || category === "Tablet"

!available
```

Operator ternar:

```
const status = available
    ? "Available"
    : "Out of stock";
```

if

```
if (price > 1000) {
    console.log("Expensive product");
} else {
    console.log("Affordable product");
}
```

for

```
for (let i = 0; i < products.length; i++) {
    console.log(products[i]);
}
```

Sintaxa moderna:

```
for (const product of products) {
    console.log(product);
}
```

switch

```
switch (category) {
    case "Electronics":
        console.log("Electronic product");
        break;

    case "Furniture":
        console.log("Furniture product");
        break;

    default:
        console.log("Unknown category");
}
```

3. Functii si Arrow Functions

O functie grupeaza logica reutilizabila.

Functie clasica

```
function calculateTax(price) {
    return price * 0.19;
}

const tax = calculateTax(1000);
```

Cu mai multi parametri:

```
function calculateTotal(price, quantity) {
    return price * quantity;
}

calculateTotal(100, 3);
```

Valori implicite

```
function calculateDiscount(price, percentage = 10) {
    return price * percentage / 100;
}

calculateDiscount(1000);
calculateDiscount(1000, 20);
```

Arrow Functions

Sintaxa ES6:

```
const calculateTax = (price) => {
    return price * 0.19;
};
```

Pentru o singura expresie:

```
const calculateTax = price => price * 0.19;
```

Mai multi parametri:

```
const calculateTotal = (price, quantity) =>
    price * quantity;
```

Arrow functions sunt folosite foarte frecvent cu metodele pentru arrays:

```
const expensiveProducts = products.filter(
    product => product.price > 3000
);
```

4. Obiecte si Arrays

Obiecte

Un obiect reprezinta o entitate cu proprietati.

```
const product = {
    id: 1,
    name: "Laptop",
    price: 3999,
    category: "Electronics",
    available: true
};
```

Accesarea proprietatilor:

```
product.name;
product.price;

product["name"];
```

Modificarea:

```
product.price = 3799;
```

Adaugarea unei proprietati:

```
product.stock = 15;
```

Metode in obiecte

Un obiect poate contine si functii.

```
const product = {
    name: "Laptop",
    price: 3999,

    getPriceWithTax() {
        return this.price * 1.19;
    }
};

product.getPriceWithTax();
```

`this` se refera aici la obiectul `product`.

Arrays

```
const products = [
    "Laptop",
    "Phone",
    "Tablet"
];
```

Accesare:

```
products[0];
products[1];
```

Lungime:

```
products.length;
```

Adaugare:

```
products.push("Monitor");
```

Stergere de la final:

```
products.pop();
```

Parcurgere:

```
for (const product of products) {
    console.log(product);
}
```

In aplicatii reale, arrays contin frecvent obiecte:

```
const products = [
    {
        id: 1,
        name: "Laptop",
        price: 3999
    },
    {
        id: 2,
        name: "Phone",
        price: 2999
    },
    {
        id: 3,
        name: "Tablet",
        price: 1999
    }
];
```

5. Destructuring si Spread/Rest Operator

Object Destructuring

In loc de:

```
const name = product.name;
const price = product.price;
```

putem scrie:

```
const { name, price } = product;
```

Putem redenumi variabilele:

```
const {
    name: productName,
    price: productPrice
} = product;
```

Valoare implicita:

```
const { discount = 0 } = product;
```

Array Destructuring

```
const products = [
    "Laptop",
    "Phone",
    "Tablet"
];

const [first, second, third] = products;
```

Putem ignora elemente:

```
const [first, , third] = products;
```

Spread Operator

Spread-ul `...` extinde valorile unui array sau obiect.

```
const products = [
    "Laptop",
    "Phone"
];

const allProducts = [
    ...products,
    "Tablet"
];
```

Pentru obiecte:

```
const product = {
    name: "Laptop",
    price: 3999
};

const updatedProduct = {
    ...product,
    price: 3799
};
```

Este foarte util pentru crearea unei copii modificate a unui obiect.

Rest Operator

Rest-ul grupeaza mai multe valori intr-un array.

```
function calculateTotal(...prices) {

    return prices.reduce(
        (total, price) => total + price,
        0
    );
}

calculateTotal(100, 200, 300);
```

6. Metode pentru Arrays

Aceste metode sunt foarte importante in JavaScript modern.

Vom utiliza:

```
const products = [
    {
        name: "Laptop",
        price: 4000,
        category: "Electronics"
    },
    {
        name: "Phone",
        price: 2500,
        category: "Electronics"
    },
    {
        name: "Desk",
        price: 800,
        category: "Furniture"
    },
    {
        name: "Chair",
        price: 500,
        category: "Furniture"
    }
];
```

forEach

Executa o functie pentru fiecare element.

```
products.forEach(product => {
    console.log(product.name);
});
```

Este folosit atunci cand vrem sa executam o actiune pentru fiecare element.

map

Transforma fiecare element si returneaza un array nou.

```
const names = products.map(
    product => product.name
```

```
);
```

Rezultat:

```
[
    "Laptop",
    "Phone",
    "Desk",
    "Chair"
]
```

Alt exemplu:

```
const pricesWithTax = products.map(
    product => product.price * 1.19
);
```

filter

Returneaza elementele care respecta o conditie.

```
const expensiveProducts = products.filter(
    product => product.price > 1000
);
```

Alt exemplu:

```
const electronics = products.filter(
    product => product.category === "Electronics"
);
```

`filter()` returneaza intotdeauna un array.

find

Returneaza primul element care respecta conditia.

```
const product = products.find(
    product => product.name === "Laptop"
);
```

Daca nu exista:

```
undefined
```

Diferenta:

```
filter() → array
find()   → un element
```

reduce

Transforma un array intr-o singura valoare.

Calcularea valorii totale:

```
const total = products.reduce(
    (sum, product) => sum + product.price,
    0
);
```

`0` este valoarea initiala a acumulatorului.

`reduce()` poate fi folosit si pentru construirea unor obiecte:

```
const summary = products.reduce(
    (result, product) => {

        result[product.category] =
            (result[product.category] || 0)
            + product.price;

        return result;
    },
    {}
);
```

Alte metode utile

Este suficient sa le recunoastem pentru moment:

```
products.some(
    product => product.price > 3000
);

products.every(
    product => product.price > 100
);

products.includes(product);

products.sort(
    (a, b) => a.price - b.price
);
```

7. Optional Chaining si Nullish Coalescing

Optional Chaining `?.`

Permite accesarea sigura a proprietatilor.

```
const user = {
    profile: {
        name: "Alex"
    }
};

console.log(user.profile?.name);
```

Daca `profile` nu exista:

```
user.profile?.name;
```

rezultatul va fi `undefined`, fara sa apara o eroare.

Poate fi folosit in lant:

```
user?.profile?.address?.city;
```

Nullish Coalescing `??`

Ofera o valoare implicita cand valoarea este `null` sau `undefined`.

```
const discount = product.discount ?? 0;
```

Exemplu:

```
const username = user.name ?? "Guest";
```

Diferenta fata de ||:

```
0 || 10       // 10
0 ?? 10       // 0

"" || "Guest" // "Guest"
"" ?? "Guest" // ""
```

8. JSON si date de la API

In aplicatiile web, datele sunt frecvent transmise folosind JSON.

Exemplu:

```
{
    "id": 1,
    "name": "Laptop",
    "price": 3999
}
```

Transformarea JSON in obiect JavaScript:

```
const product = JSON.parse(jsonString);
```

Transformarea unui obiect in JSON:

```
const json = JSON.stringify(product);
```

Exemplu:

```
const product = {
    id: 1,
    name: "Laptop",
    price: 3999
};

const json = JSON.stringify(product);

console.log(json);
```

In aplicatiile web, datele primite de la backend sunt de obicei prelucrate sub forma de obiecte si arrays.

9. Asincronism si Promises

Operatiile precum request-urile HTTP sau accesarea unor servicii externe pot dura un anumit timp.

JavaScript nu blocheaza executia intregului program in timp ce asteapta.

Exemplu:

```
console.log("Start");

setTimeout(() => {
    console.log("Finished");
}, 2000);

console.log("End");
```

Rezultatul:

```
Start
End
Finished
```

Promise

Promise reprezinta rezultatul unei operatii care se va finaliza in viitor.

Promise poate fi:

```
pending
   ↓
fulfilled

sau

pending
   ↓
rejected
```

Exemplu:

```
const promise = new Promise(
    (resolve, reject) => {

        const success = true;

        if (success) {
            resolve("Operation completed");
        } else {
            reject("Operation failed");
        }
    }
);
```

Consumarea Promise:

```
promise
    .then(result => {
        console.log(result);
    })
    .catch(error => {
        console.error(error);
    });
```

Exemplu cu `setTimeout`

Simulam un request catre backend:

```
function fetchProductsFromDB() {

    return new Promise((resolve, reject) => {

        setTimeout(() => {

            const products = [
                {
                    name: "Laptop",
                    price: 4000
                },
                {
                    name: "Phone",
                    price: 2500
                }
            ];

            resolve(products);

        }, 1000);
    });
}
```

Utilizare:

```
fetchProductsFromDB()
    .then(products => {
        console.log(products);
    })
    .catch(error => {
        console.error(error);
    });
```

10. `async` / `await` si tratarea erorilor

`async` / `await` ofera o sintaxa mai simpla pentru lucrul cu Promise.

```
async function loadProducts() {

    const products =
        await fetchProductsFromDB();

    console.log(products);
}
```

`await` asteapta rezultatul Promise-ului.

`try` / `catch`

Pentru tratarea erorilor:

```
async function loadProducts() {

    try {

        const products =
            await fetchProductsFromDB();

        console.log(products);

    } catch (error) {

        console.error(error);
    }
}
```

Fluxul este:

```
async function
      ↓
    await
      ↓
   Promise
   ↙     ↘
success   error
  ↓         ↓
result    catch
```

In aplicatiile reale, acest pattern este foarte frecvent pentru request-uri catre API-uri.

11. Module ES6

Modulele permit impartirea aplicatiei in mai multe fisiere.

helpers.js

```
export function calculateTax(price) {
    return price * 0.19;
}

export function formatPrice(price) {
    return `${price} RON`;
}
```

main.js

```
import {
    calculateTax,
    formatPrice
} from "./helpers.js";

console.log(calculateTax(1000));
console.log(formatPrice(1000));
```

Default Export

Un fisier poate avea un export implicit:

```
export default function calculateDiscount(price) {
    return price * 0.1;
}
```

Import:

```
import calculateDiscount
    from "./helpers.js";
```

Diferenta:

```
export function test() {}
```

se importa folosind numele exportat:

```
import { test } from "./file.js";
```

In cazul:

```
export default function test() {}
```

numele poate fi ales la import:

```
import calculate from "./file.js";
```